

第3章 流程控制

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 `if` 的初始化语句与"条件必须是布尔"的约束
- 理解 `switch` 在 Go 中的三大不同:没有 **fallthrough**(默认隐式 `break`)、表达式 `switch`、空表达式 `switch` 做多条件分支
- 掌握 `for` 是唯一循环:经典三段式、条件式(替代 `while`)、无限循环、`range` 遍历
- 理解 `range` 遍历的**值拷贝**语义与按下标修改的正确姿势
- 掌握带标签的 `break` / `continue` 跳出多层嵌套
- 理解 `goto` 在 Go 中的合法用途(统一错误出口)与限制
- 掌握循环变量捕获 bug:Go 1.22 前的经典陷阱与 1.22 的语义修复

💡 从 C 到 Go:本章主题如何迁移

C 的流程控制关键词 Go 全有——`if`、`else`、`switch`、`for`、`break`、`continue`、`goto`——但行为差异密集,堪称"形似而神不似"的一章。

最大的差异在 `switch`。C 的 `switch` 只接受整型、默认贯穿(fallthrough),忘写 `break` 是经典 bug 来源;Go 的 `switch` 隐式 `break`(想贯穿必须显式写 `fallthrough`),而且条件可以是任意可比较类型,甚至可以不写条件表达式——变成"多条件 `if` 链"的漂亮写法。C 程序员写 Go 时常见的两个动作:一是老想加 `break` (无害,Go 允许冗余 `break`),二是老忘 `fallthrough` 才导致行为差异(有害)。

循环的差异是"收敛":C 有 `for` / `while` / `do-while` 三种,Go 只有 `for` 一种,`while` 用 `for` 条件 表达,无限循环用 `for {}`。少了语法,多了统一。而 `range` 遍历则是 C 完全没有的:遍历数组/切片/字符串/map/通道,同时给出下标与值——但注意**值是拷贝**,修改元素必须用下标。

还有一处历史教训:Go 1.22 之前, `for` 循环变量在每次迭代中共享同一份存储,闭包捕获它会出现"全部读到最后一个值"的经典 bug(与 C 的悬垂、JS 的 `var` 同病);Go 1.22 起每次迭代的变量都是新的,语义正确且兼容。

一句话主线: C 的流程控制是"放任自由"(fallthrough、隐式真值、无约束),Go 的流程控制是"替你把该做的都做了"(隐式 `break`、强制布尔、标签跳转)。

📦 前置知识自查

- C 的 `if/else`、`switch`(含 fallthrough 行为)、`for/while/do-while`、`break/continue/goto`
- C 的"非零即真"条件语义与短路求值
- 数组遍历的经典下标写法 `for (i = 0; i < n; i++)`
- `switch` 只能整型/枚举的局限(不能比较字符串、浮点)

本章学习路线

1. **第一阶段:**§3.1–§3.2,重点吃透 switch 的三个不同点,做几道 switch 改写题
2. **第二阶段:**§3.3–§3.5,掌握三种 for 形态与 range 值拷贝、标签跳转
3. **第三阶段(实验):**§3.6–§3.7 与章末实验:菜单程序 + 素数筛选,把全部控制流织进一个程序
4. **验收标准:**能不看资料解释"switch 不写 break 会怎样"与"range 修改元素为什么无效"

本章知识导图

- §3.1 if/else:初始化语句、强制布尔条件、悬挂 else 的消失
- §3.2 switch:隐式 break、表达式 switch、空表达式 switch、fallthrough
- §3.3 for 的三种形态:三段式、条件式、无限循环
- §3.4 range 遍历:数组/切片/字符串/map/通道、值拷贝
- §3.5 break/continue 与标签:跳出多层嵌套
- §3.6 goto 的合法用途与限制
- §3.7 循环变量捕获:1.22 前的经典 bug 与修复

§3.1 if/else:初始化语句与强制布尔

Go 的 `if` 与 C 的三处差异:

- **条件不带括号**(`if x > 0` 而非 `if (x > 0)`)
- **条件必须是布尔**:没有"非零即真",`if x` 当 `x` 是 int 时编译错误
- **可以有初始化语句**:`if err := f(); err != nil`,变量作用域限于该 if-else 块内——这是 Go 最常用的惯用法,把"调用、判错、处理"收敛到一行

关于**悬挂 else**:C 的 `if (a) if (b) x; else y;` 中 `else` 就近匹配 `if (b)`,易错。Go 要求 `else` 必须与同一行的 `}` 紧邻(`} else {`),从语法层面消灭了歧义。

■ 示例 3-1 `if` 的初始化语句与强制布尔

```

package main

import (
    "fmt"
    "strconv"
)

func main() {
    // 初始化语句:err 只在 if-else 内可见
    if s, err := strconv.Atoi("42"); err == nil {
        fmt.Println("解析成功:", s)
    } else {
        fmt.Println("解析失败:", err)
    }
    // 此处访问 s 或 err 是编译错误

    // 条件必须是布尔:下面两行都是编译错误
    // n := 1
    // if n { fmt.Println("非零") }

    // else 必须紧跟 "}" 同行,悬挂 else 不可能出现
    if 1 > 2 {
        fmt.Println("不会执行")
    } else {
        fmt.Println("必然执行")
    }
}

```

工程建议:用"提前返回"减少嵌套

C 风格是深嵌套 if 金字塔;Go 风格是每层校验后直接 `return`,把正常路径放平。写多条件校验时,坚持"先失败先返回",函数主体自然保持在缩进两级以内——这在 Go 社区是被强制 code review 的惯例。

§3.2 switch:没有 fallthrough 的 C 对照

C 的 `switch` 的缺陷:只支持整型、默认贯穿、忘写 `break` 就掉进下一个 case。Go 的 `switch` 全部重做:

C 的写法	Go 的写法
<code>switch (x) { case 1: ...; break; ... }</code> ,忘写 <code>break</code> 会贯穿	<code>switch x { case 1: ... }</code> ,每个 case 自动 <code>break</code>
只能整型/枚举(字符、字符串要 if-else 链)	任何可比较类型:字符串、结构体、接口均可
case 必须是常量表达式	case 可以是任意表达式,按顺序求值匹配
多条件写 <code>case 1: case 2:</code> 依赖贯穿	多条件写 <code>case 1, 2:</code> 用逗号分隔

示例 3-2 switch 的现代形态

```

package main

import "fmt"

func scoreLevel(score int) string {
    switch {
    case score >= 90:
        return "优秀"
    case score >= 60:
        return "及格"
    default:
        return "不及格"
    }
}

func main() {
    // 表达式 switch:匹配字符串
    day := "周一"
    switch day {
    case "周一", "周二", "周三", "周四", "周五":
        fmt.Println("工作日")
    case "周六", "周日":
        fmt.Println("休息日")
    default:
        fmt.Println("未知")
    }

    // 空表达式 switch:等价于多条件 if 链(上面的 scoreLevel)
    fmt.Println(scoreLevel(95), scoreLevel(75), scoreLevel(40))

    // 表达式 switch 的 case 按书写顺序求值
    n := 5
    switch n {
    case 5:
        fmt.Println("先命中 n == 5")
        // 不写 break 也不会继续执行下面的 case
    case 5:
        fmt.Println("不可达:同一个 switch 里重复的值")
    }

    // fallthrough:强制贯穿到下一个 case(罕见,明确标注)
    switch n {
    case 5:
        fmt.Println("命中 5,并强制贯穿")
        fallthrough
    case 6:
        fmt.Println("被贯穿执行")
    }
}

```

⚠ 易错点 3-1:fallthrough 的语义是"无条件执行下一个 case"

`fallthrough` 不重新判断条件,直接进入下一个 case 的语句体——即使下一个 case 的条件是 `false`。它不能放在一个 case 的最后一条语句之外的位置,也不能贯穿到下一个含初始化的 case(编译错误)。生产代码里 `fallthrough` 非常罕见,看到它先怀疑是否应该用逗号多条件。

⚠ 易错点 3-2:switch 空表达式不等于 if-else 的短路顺序

`switch { case A: ... case B: ... }` 从上到下依次判断,命中即停,这点与 if-else 链一致。但注意 case 中不要写有副作用的调用并依赖"只求值一次"——每个 case 只在其轮到判断时求值,顺序敏感,与 C 的 `switch` 完全不同。

§3.3 for 的三种形态:唯一的循环

Go 没有 `while`、没有 `do-while`,只有 `for`,三种形态:

C 的写法	Go 的写法
<code>for (i = 0; i < n; i++)</code>	<code>for i := 0; i < n; i++</code> (分号分隔,条件无括号)
<code>while (cond)</code>	<code>for cond</code> (省略分号)
<code>for (;;) </code>	<code>for {}</code> (省略全部,直接写空块)
<code>do { } while(cond)</code>	无此语法;用 <code>for { ...; if !cond { break } }</code> 模拟

示例 3-3 for 的三种形态

```

package main

import "fmt"

func main() {
    // 形态一:经典三段式(初始化语句的变量只属于循环)
    for i := 0; i < 3; i++ {
        fmt.Print(i)
    }
    fmt.Println()

    // 形态二:条件式(替代 while)
    n := 0
    for n < 3 {
        n++
    }
    fmt.Println("n =", n)

    // 形态三:无限循环 + break(替代 do-while 的常用模式)
    i := 0
    for {
        i++
        if i >= 3 {
            break // 先执行后判断,语义接近 do-while
        }
    }
    fmt.Println("i =", i)

    // 空初始化、条件、步进都可以省略一部分
    sum := 0
    for i := 0; ; i++ {
        if i > 10 {
            break
        }
        sum += i
    }
    fmt.Println("sum =", sum)
}

```

§3.4 range 遍历:值拷贝与正确修改

`for i, v := range collection` 是 Go 的特色遍历,支持数组、切片、字符串、map、通道。两个关键点:

- `i` 是下标(字符串中是字节下标), `v` 是元素的**拷贝**—— `v = ...` 不会影响原集合,修改必须用 `s[i] = ...`
- 不需要值或不需要下标时,用 `_` 占位: `for _, v := range s` 或 `for i := range s`
- map 的遍历顺序**随机**,不要依赖任何顺序;需要顺序就先把键排序再遍历

示例 3-4 range 遍历:值是拷贝

```

package main

import "fmt"

func main() {
    nums := []int{10, 20, 30}

    // 尝试修改副本:无效
    for _, v := range nums {
        v = 0
    }
    fmt.Println(nums) // [10 20 30]:没变!

    // 用下标修改:有效
    for i := range nums {
        nums[i] *= 2
    }
    fmt.Println(nums) // [20 40 60]

    // 字符串:range 按 rune 遍历,下标是字节位置
    for i, r := range "Hi 好" {
        fmt.Printf("i=%d r=%q\n", i, r)
    }

    // map:遍历顺序随机
    m := map[string]int{"a": 1, "b": 2}
    for k, v := range m {
        fmt.Println(k, v)
    }

    // 通道:range 持续接收直到通道关闭(第 9 章详解)
    ch := make(chan int, 3)
    ch <- 1
    ch <- 2
    ch <- 3
    close(ch)
    for v := range ch {
        fmt.Print(v)
    }
    fmt.Println()
}

```

⚠ 易错点 3-3:遍历切片时 append 会不会无限循环?

对切片 `for i := range s { s = append(s, x) }`:`range` 在进入循环时只求值一次(取当时的长度),后续 `append` 改的是变量 `s` 而非迭代器,所以循环次数固定、不会无限,但也不会遍历到新增元素。若真的需要"消费式"遍历(边读边清),改用显式下标循环并控制长度。

§3.5 break/continue 与标签

Go 的 `break / continue` 默认只作用于最近一层循环(或 `switch`)。要跳出/继续**外层**循环,给循环加标签,然后 `break` 标签 或 `continue` 标签——这是 C 程序员最需要的升级:在 C 里你只能靠 `goto` 或标志位,Go 给了结构化的命名跳转。

示例 3-5 带标签的 break/continue:矩阵搜索

```
package main

import "fmt"

func main() {
    // 在矩阵中找目标值,找到后跳出两层循环
    matrix := [][]int{
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9},
    }
    target := 8

    outer: // 标签:必须紧贴循环
        for i, row := range matrix {
            for j, v := range row {
                if v == target {
                    fmt.Printf("找到 %d,位置 (%d,%d)\n", target, i, j)
                    break outer // 跳出两层
                }
            }
        }

    // continue 带标签:跳过外层循环的本次迭代
    count := 0
loop:
    for i := 0; i < 3; i++ {
        for j := 0; j < 3; j++ {
            if i+j > 2 {
                continue loop // 直接进入外层下一次迭代
            }
            count++
        }
    }
    fmt.Println("满足 i+j<=2 的对数:", count)
}
```



工程建议:标签命名用"意图"而不是位置

标签名建议体现循环的职责,如 `readLoop:`、`retryLoop:`,而不是 `L1:`。标签只在少数"多层退出"场景使用;如果能用提前 `return` 重构,优先 `return`——标签嵌套超过两层时,读者会开始头疼。

§3.6 goto 的合法用途与限制

C 的 `goto` 名声狼藉(语句式 spaghetti)。Go 保留了 `goto`,但加了严格限制:不能跳进变量作用域(会编译错误)、不能跳过初始化语句、只能向后或向前跳(同函数内)。因此 Go 的 `goto` 反而安全——它的合理用途是**统一错误出口**:

示例 3-6 goto 的统一错误清理(经典模式)


```

package main

import "fmt"

// 模拟三个可能失败的步骤
func step1() error { fmt.Println("步骤1"); return nil }
func step2() error { fmt.Println("步骤2"); return nil }
func step3() error { return fmt.Errorf("步骤3 失败了") }

func run() (err error) {
    // 失败后集中清理并返回,避免在每个分支重复写
    if err = step1(); err != nil {
        goto fail
    }
    if err = step2(); err != nil {
        goto fail
    }
    if err = step3(); err != nil {
        goto fail
    }
    fmt.Println("全部成功")
    return nil
}

fail:
    fmt.Println("清理现场...")
    return err
}

func main() {
    if err := run(); err != nil {
        fmt.Println("run 返回错误:", err)
    }
}

```

⚠ 易错点 3-4: goto 不能跳入变量作用域

Go 编译器会拒绝“向前跳入一个新变量已声明的作用域”的 goto。这在实践中是保护——它让 goto 不可能成为 C 时代那种“跳进初始化中间”的杀手。同样, goto 只能在同一函数内。多数情况下,提前 return + 延迟调用 (defer, 第 4 章)比 goto 更简洁,先学会 defer 再决定用哪个。

§3.7 循环变量捕获:1.22 前的经典 bug 与修复

这是 Go 历史上被引用最多的 bug 之一。Go 1.22 之前, `for i := 0; ...` 与 `for _, v := range` 中,循环变量在整个循环里只有一份存储,每次迭代复用。如果闭包(第 4 章)捕获了 `v`,所有闭包共享同一个变量——循环结束后它们读到的都是最后一次迭代的值。

■ 示例 3-7 循环变量捕获:1.21 与 1.22 的行为对比

```

package main

import "fmt"

func main() {
    // 经典 bug:启动 3 个"任务",期望打印 0、1、2
    funcs := []func(){}
    for i := 0; i < 3; i++ {
        funcs = append(funcs, func() { fmt.Print(i) })
    }
    fmt.Print("旧语义(1.21 及之前):")
    for _, f := range funcs {
        f()
    }
    fmt.Println(" — 全部打印 3(变量共享)")

    // 修复写法(1.21 时代):复制一份到循环体内
    funcs2 := []func(){}
    for i := 0; i < 3; i++ {
        i := i // 局部副本
        funcs2 = append(funcs2, func() { fmt.Print(i) })
    }
    fmt.Print("经典修复(循环内复制):")
    for _, f := range funcs2 {
        f()
    }
    fmt.Println()

    // Go 1.22 起:每次迭代的循环变量都是新变量,上面两个写法等价
    funcs3 := []func(){}
    for i := 0; i < 3; i++ {
        funcs3 = append(funcs3, func() { fmt.Print(i) })
    }
    fmt.Print("新语义(1.22+):")
    for _, f := range funcs3 {
        f()
    }
    fmt.Println(" — 正确输出 0 1 2")
}

```

★ 重点:Go 1.22 的循环变量语义

Go 1.22 起, `for` 与 `range` 的循环变量在**每次迭代重新声明**,闭包捕获到的是各自迭代的变量。生产要点:如果团队代码要兼容 Go 1.21,坚持"循环体内第一行 `i := i`"的经典写法——它在两个版本下都正确;只跑 1.22+ 则可以删掉冗余副本。`go.mod` 中的 `go 1.22` 指令会让编译器采用新语义。

本章速查表

主题	要点
if 语法	<code>if 条件 { } else { }</code> ; 条件无括号、必须是布尔; 可带初始化语句
悬挂 else	不存在: <code>else</code> 必须与 <code>}</code> 同行
switch	隐式 <code>break</code> , 无贯穿; <code>case 1, 2:</code> 多条件; <code>case</code> 可为任意表达式
空表达式 switch	<code>switch { case cond1: ... }</code> 等价多条件 if 链, 从上到下命中即停
fallthrough	强制执行下一个 <code>case</code> (不判断条件); 生产罕见
for 三段式	<code>for i := 0; i < n; i++</code> ; 初始化变量仅循环内可见
while 替代	<code>for 条件 { }</code> ; <code>do-while</code> 用 <code>for { ...; if !c { break } }</code>
无限循环	<code>for { }</code> 或 <code>for true { }</code> ; 配合 <code>break</code>
range 遍历	<code>for i, v := range 集合; _</code> 占位跳过; <code>for i := range s</code> 只取下标
range 值拷贝	<code>v</code> 是副本; 修改元素用 <code>s[i] = ...</code>
map 遍历顺序	随机; 需要顺序先排序键
标签跳转	<code>outer:</code> 紧贴循环; <code>break outer</code> / <code>continue outer</code>
goto	合法(统一错误出口); 不能跳入变量作用域; 同函数内
循环变量捕获	1.22 起每迭代新变量; 兼容 1.21 用 <code>i := i</code> 副本

最小必会清单

- 能写出带初始化语句的 `if`, 并解释变量作用域
- 能解释 Go 的 `switch` 为什么不写 `break` 也不会贯穿"与 `fallthrough` 的精确语义
- 能把一段 C 的 `while/do-while` 改写为 Go 的 `for`
- 能解释 `range` 遍历中为什么 `v = ...` 无效、`s[i] = ...` 有效
- 能写出带标签的 `break` 跳出两层循环的完整代码
- 能说出 `goto` 在 Go 中的两个限制和一个合法用途
- 能解释循环变量捕获 bug 的成因、1.22 的变化与兼容写法
- 能写出 `map` 排序遍历的标准三段式(取键、排序、遍历)

自测题

Q 1. 下面的 Go 代码输出什么? `switch 2 { case 1, 2: fmt.Print("A"); case 2: fmt.Print("B") }`

✅ 参考答案

输出 "A"。 `case 1, 2` 命中 2 后执行, `case` 结束自动 `break`, 不会继续判断第二个 `case` (即使它也是 2)。Go 的 `case` 从上到下第一个命中即停。

Q 2. 用 Go 重写下面的 C 代码: `while (n < 100) { n *= 2; }` 和 `do { n /= 2; } while (n > 1);`

✓ 参考答案

`for n < 100 { n *= 2 }; for { n /= 2; if n <= 1 { break } }`。注意 do-while 是"先执行后判断",因此用无限循环 + 内部判断模拟,判断条件取反。

Q 3. `s := []int{1,2,3}; for _, v := range s { v += 10 }; fmt.Println(s)` 输出什么?为什么?

✓ 参考答案

输出 [1 2 3]。range 的 v 是元素拷贝, `v += 10` 只修改副本。必须写 `for i := range s { s[i] += 10 }` 才能修改原切片。

Q 4. 在 Go 1.21 下,下面代码的打印结果是什么? `for i := 0; i < 3; i++ { go func() { fmt.Print(i) }() }` (假设有等待机制)

✓ 参考答案

极大概率打印 "333"(三个 goroutine 共享同一个 i,循环结束时 `i == 3`)。在 Go 1.22+ 下打印 "012"(顺序不定,但值是 0、1、2)。这就是循环变量捕获 bug。修复:循环体内写 `i := i` 或把 i 作为参数传入闭包。

Q 5. goto 在 Go 中哪些行为是编译错误?(a) 向后跳;(b) 跳入一个含新变量的块;(c) 跳到另一个函数;(d) 跳过一个变量的初始化后使用该变量。

✓ 参考答案

(a) 合法,向后跳允许;(b) 编译错误,禁止跳入变量作用域;(c) 编译错误,goto 限同一函数;(d) 编译错误,会跳过初始化导致"变量未初始化"检查失败。

Q 6. 为什么说"range map 时不要依赖遍历顺序"?写出按 key 排序遍历 map 的惯用法。

✓ 参考答案

Go 规范明确 map 遍历顺序未定义且故意随机化(防止程序员依赖)。惯用法:先 `keys := make([]string, 0, len(m))` 收集键, `sort.Strings(keys)`,再 `for _, k := range keys { v := m[k] ... }`。

章末实验题:菜单驱动的控制台程序 + 素数筛选

实验 3:文本菜单 + 素数筛选 + 数组统计

实验目标:实现一个菜单驱动的控制台工具:维护一组整数数据,提供"输入数据、素数筛选、数组统计、退出"四个功能,强制使用本章流程控制知识点。

实验要求:

- 任务 1:主循环用 `for {}` 无限循环 + 菜单读取,退出用带标签的 `break` 跳出(覆盖:§3.3 无限循环、§3.5 标签)
- 任务 2:菜单分发用 `switch` 表达式(字符串条件),无效选项进 `default`(覆盖:§3.2 `switch`)
- 任务 3:输入数据功能:读一行、按空格切分、逐项 `strconv.Atoi`,非法输入用 `continue` 跳过并提示(覆盖:§3.1 `if`、§3.3 `for` 三段式、§3.4 遍历)
- 任务 4:素数筛选用埃拉托色尼筛法:外层 `for i := 2; i*i <= n; i++`,内层从 `i*i` 起步(覆盖:§3.3 两种 `for` 形态)
- 任务 5:数组统计遍历用 `range`,最大值/最小值/偶数的判断用 `if-else`(覆盖:§3.4、§3.1)
- 任务 6:退出功能同时用 `return` 与带标签 `break` 两种写法各实现一次并对比(覆盖:§3.5)

实验环境:Go 1.21+, `go run main.go`,用 `go vet` 静态检查。

第一步 写数据模型:一个 `[]int` 切片,注意用 `data[:0]` 复用底层数组清空(回顾第 1 章切片)

第二步 现 `sieve` 函数:布尔数组标记合数,注意 `make([]bool, n+1)` 与下标边界

第三步 现 `stats` 函数:`range` 遍历求和、求最值、统计偶数;平均分输出保留一位小数(回顾第 2 章格式化)

第四步 主菜单循环:`switch` 分发四个 `case`,退出 `case` 用带标签 `break`;再改成 `return` 版本对比

第五步 行并验证:输入 2 3 5 7 11 做素数筛选与统计,确认输出正确

✔ 参考答案要点

```
// 菜单驱动的工具:覆盖 if/switch/三种 for/标签跳转
package main

import (
    "bufio"
    "fmt"
    "os"
    "strconv"
    "strings"
)

// sieve 埃拉托色尼筛法:返回 [0,n] 内全部素数
func sieve(n int) []int {
    if n < 2 {
        return []int{}
    }
    notPrime := make([]bool, n+1) // 零值 false:全部先当素数
    notPrime[0], notPrime[1] = true, true
    for i := 2; i*i <= n; i++ { // 经典三段式 for
        if notPrime[i] {
            continue // 已排除,跳过(continue 用法)
        }
        for j := i * i; j <= n; j += i { // 从 i*i 起步
            notPrime[j] = true
        }
    }
    primes := []int{}
    for i := 2; i <= n; i++ {
        if !notPrime[i] {
            primes = append(primes, i)
        }
    }
    return primes
}

// stats 统计数组:和/平均/最大/最小/偶数个数
func stats(nums []int) {
    if len(nums) == 0 { // 提前返回:空数组
        fmt.Println("数组为空,无法统计")
        return
    }
    sum := 0
    max, min := nums[0], nums[0]
    evens := 0
    for _, v := range nums { // range 遍历
        sum += v
        if v > max {
            max = v
        }
        if v < min {
            min = v
        }
        if v%2 == 0 {
            evens++
        }
    }
}
```

```

    }
    fmt.Printf("和=%d 平均=%.1f 最大=%d 最小=%d 偶数=%d 个\n",
        sum, float64(sum)/float64(len(nums)), max, min, evens)
}

func main() {
    scanner := bufio.NewScanner(os.Stdin)
    data := []int{}
    fmt.Println("==== 菜单驱动的工具 ====")

mainLoop: // 标签:退出用带标签 break
    for {
        fmt.Println("1. 输入数据 2. 素数筛选 3. 数组统计 4. 退出")
        fmt.Print("请选择: ")
        scanner.Scan()
        switch choice := scanner.Text(); choice { // 表达式 switch + 初始化语句
        case "1":
            fmt.Print("输入整数,空格分隔: ")
            scanner.Scan()
            fields := strings.Fields(scanner.Text())
            data = data[:0] // 清空旧数据,复用底层数组
            for _, f := range fields {
                v, err := strconv.Atoi(f)
                if err != nil { // 非法输入:continue 跳过
                    fmt.Println("忽略非法输入:", f)
                    continue
                }
                data = append(data, v)
            }
            fmt.Println("当前数据:", data)
        case "2":
            fmt.Print("输入上限 n: ")
            scanner.Scan()
            n, err := strconv.Atoi(scanner.Text())
            if err != nil || n < 0 {
                fmt.Println("非法输入")
                continue
            }
            primes := sieve(n)
            fmt.Printf("%d 以内素数 %d 个:%v\n", n, len(primes), primes)
        case "3":
            stats(data)
        case "4":
            fmt.Println("再见")
            break mainLoop // 带标签跳出主循环
        default: // switch 的 default 分支
            fmt.Println("无效选项")
        }
    }
}

```

设计说明:主循环 `for {}` 配合标签 `mainLoop`,退出项用 `break mainLoop` 一步跳出;菜单用带初始化语句的表达式 `switch(choice` 只在 `switch` 内可见);筛法用两种 `for` 形态嵌套,`continue` 跳过已标记数;统计用 `range` 与多重

if。全部知识点织入约 90 行代码。扩展练习:把"清空数据"改为 `data = data[:0]` 与 `data = nil` 两种,比较行为差异(第 14 章详解 `nil` 切片)。

下一章预告:第4章 函数与方法。流程控制就位后,轮到代码的积木:多返回值 `(T, error)` 是 Go 的灵魂、`defer` 的资源管理、闭包与函数值、可变参数—从 C 的函数指针时代全面升级。